Library Database Management System

Requirements Document — Project Part 2

Team Lead: Parker | Date: September 24, 2026

---

1. Introduction

1.1 Project Overview

The purpose of this project is to design and implement a relational database system to support a library. The database will serve as the central repository for information about the library's collection, its members, and its staff. It will track the full lifecycle of an item, from acquisition and cataloging through checkout, check-in, and reservation, and it will support the reporting needs of library staff. The system will be accessed through a web-based user interface with two distinct views. One for members and one for administration.

1.2 Scope

This project will begin with a core set of features and expand outward as time permits. The base functionality that must be delivered includes: a working administrative web interface, a working member web interface, a populated database filled with media items, and a basic set of queries covering search, insertion, and deletion of records. Stretch goals beyond this base include richer reporting, reservation/hold management, and additional search filters for end users.

1.3 Glossary

Term / Acronym	Definition
Media Item	Any physical item in the library's circulating collection that can be checked out.
Member	A registered library patron who can search the catalog and check out, check in, or reserve items.
Admin	A staff user with elevated privileges to add, edit, or remove records and generate reports.
Checkout	The process of a member borrowing a media item, recorded with a due date.
Check-in	The process of returning a previously checked-out item to the library.
Reservation	A request placed by a member to reserve an item that is currently checked out.

Term / Acronym	Definition
UI	User Interface — the web-based front end through which users interact with the database.
CRUD	Create, Read, Update, Delete — the four basic database operations.
ORM	Object-Relational Mapping — a technique/library used to interact with the database from application code.

2. Stakeholders

The following individuals and groups have an interest in, or are affected by, the design and operation of the library database:

Stakeholder	Role / Interest
Library Members (Clients)	Everyday patrons who search the catalog, check out and return items, and place reservations. They need fast, accurate search results but do not have permission to modify catalog or circulation data.
Librarians / Staff (Admins)	Employees responsible for maintaining the collection: adding and removing items, processing checkouts and check-ins, managing member records, and generating operational reports.
Library Director / Management	Uses aggregate reports (e.g., circulation trends, overdue items, collection usage) to make budgeting and acquisition decisions. Contrived for this project to represent a realistic reporting audience.
System Administrator (Caleb)	Responsible for creating, hosting, and administering the database and the GitHub repository; manages backups, access control, and deployment.
Development Team	Designs, builds, tests, and documents the database and its web interfaces.

3. Requirements

3.1 Functional Requirements

The database and its supporting application must support the following functions, grouped by category:

User Administration

- Members and admins must log in with a username/email and password; the system must distinguish between the two roles and route each to the appropriate UI view.
- Admins may create, edit, deactivate, or delete member and staff accounts
- Members may view and update their own contact information but cannot change their role or permissions

Data Entry, Retrieval, Updates, and Deletions

- Admins can insert new media items into the catalog, including title, author/publisher, and classification details.
- Admins can update existing catalog records and delete items that are withdrawn from circulation
- Members and admins can search/retrieve items by title, author, subject, ISBN, or availability status
- Members can view their own checkout history and current holds. Admins can view any member's history.

Circulation

- The system must support checking out an item to a member, recording the checkout data and calculated due date
- The system must support checking in a returned item, updating its availability and clearing the associated checkout record
- The system must support placing a reservation on an item that is currently checked out, and must notify when an item becomes available

Report Generation

The following queries are required for this project:

1) List all currently available copies of a given title
2) Insert a new media item into the catalog
3) Delete a media item from the catalog
4) List all items currently checked out by a given member
5) List all overdue items and members who hold them
6) Generate a circulation report showing checkout counts per item over a given date range
7) List all items with an active reservation sorted by hold date
8) List all magazines by issue date for a given publication
9) Report the total number of items in the collection by category or media type
10) Identify members with no activity in the past N months

3.2 Data Entities

The following entities represent the major objects in the system. Attributes may be refined or expanded during conceptual (ER) modeling.

Media Item (base entity for all circulating items)

Attribute	Data Type / Constraint
item_id	INTEGER, Primary Key, auto-increment
title	VARCHAR(255), NOT NULL
media_type	VARCHAR(50), e.g., 'Book', 'Magazine' (NOT NULL)
author_or_publisher	VARCHAR(255)
isbn_or_issn	VARCHAR(20), UNIQUE where applicable
publication_date	DATE

Attribute	Data Type / Constraint
status	VARCHAR(20), e.g., 'Available', 'Checked Out', 'Reserved', 'Withdrawn'
date_added	DATE, NOT NULL, default current date

Magazine (specialization of Media Item)

Attribute	Data Type / Constraint
item_id	INTEGER, Primary Key, Foreign Key to Media Item
issue_number	VARCHAR(20)
volume	VARCHAR(20)
frequency	VARCHAR(20), e.g., 'Weekly', 'Monthly'

Client / Member

Attribute	Data Type / Constraint
client_id	INTEGER, Primary Key, auto-increment
first_name	VARCHAR(100), NOT NULL
last_name	VARCHAR(100), NOT NULL
email	VARCHAR(255), UNIQUE, NOT NULL
password_hash	VARCHAR(255), NOT NULL
phone	VARCHAR(20)
membership_date	DATE, NOT NULL
account_status	VARCHAR(20), e.g., 'Active', 'Suspended'

Admin / Librarian

Attribute	Data Type / Constraint
admin_id	INTEGER, Primary Key, auto-increment
first_name	VARCHAR(100), NOT NULL
last_name	VARCHAR(100), NOT NULL
email	VARCHAR(255), UNIQUE, NOT NULL
password_hash	VARCHAR(255), NOT NULL
role	VARCHAR(50), e.g., 'Librarian', 'System Admin'
hire_date	DATE

Checkout / Circulation Record

Attribute	Data Type / Constraint
checkout_id	INTEGER, Primary Key, auto-increment
item_id	INTEGER, Foreign Key to Media Item, NOT NULL
client_id	INTEGER, Foreign Key to Client, NOT NULL
checkout_date	DATE, NOT NULL
due_date	DATE, NOT NULL
return_date	DATE, NULL until returned

Reservation / Hold

Attribute	Data Type / Constraint
reservation_id	INTEGER, Primary Key, auto-increment
item_id	INTEGER, Foreign Key to Media Item, NOT NULL
client_id	INTEGER, Foreign Key to Client, NOT NULL
reservation_date	DATE, NOT NULL
status	VARCHAR(20), e.g., 'Pending', 'Fulfilled', 'Cancelled'

4. Hardware and Software Requirements

This database will be hosted in the cloud rather than run as a fully standalone local instance

4.1 Software

- Database Engine: PostgreSQL, hosted on Neon (serverless Postgres platform).
- Database Access: Application code connects to the database via standard PostgreSQL client libraries/drivers over a secure connection URL provided by Neon.
- Web Application: A web-based front end providing separate Admin and User (member) interfaces, communicating with the database through a backend application layer.
- Version Control: GitHub repository for source code, schema scripts, and documentation.
- Development Environment: Any modern code editor/IDE capable of running the chosen backend language/framework and connecting to PostgreSQL.

4.2 Hardware

- Server-side: No dedicated physical hardware is required; the database runs on Neon's managed cloud infrastructure, which handles compute and storage scaling.
- Client-side: Any laptop or desktop capable of running a modern web browser and, for development, a local development environment (Node.js/Python/etc. as chosen by the team) with internet access to reach the Neon-hosted database.

- Minimum recommended client specs for development: 8 GB RAM, modern multi-core CPU, and a stable internet connection.

5. Appendices

5.1 GitHub Repository Management

All project artifacts, including schema definition scripts, sample data, application source code, and documentation such as this requirements document, will be maintained in the team's GitHub repository. Caleb is responsible for creating, running, and administering the repository, including managing branches and access for team members. The team lead will submit the repository URL for this project part on Canvas.

5.2 Revision Notes

This document builds on Project Part 1 and will continue to evolve as the conceptual (ER) model and physical schema are developed. Data entities and attributes listed in Section 3.2 are expected to be expanded during ER modeling.